

Overall Goals: keep GTSAM reliable on the platforms we each build against, and make it easier to learn, that includes real documentation for MATLAB & runnable Python examples in notebooks.

PROJECT OVERVIEW

GTSAM (Georgia Tech Smoothing and Mapping) is a C++ library for factor-graph-based optimization in robotics and vision. It acts as the optimization core behind modern simultaneous localization and mapping (SLAM) & sensor fusion, exposed to users through Python and MATLAB wrappers. This summer we worked to make that whole stack more reliable and more accessible: Grant on the MATLAB wrapper and Windows build, Jiahao on the Python/C++ ecosystem and runnable examples.

GRANT: MATLAB WRAPPER & WINDOWS BUILD

10 issues closed · 5 PRs merged (gtsam, wrap) · #80 active through Friday · also triaged #2489, #2485, & more

Fixed Windows build blockers spanning all three languages: Python, C++, and MATLAB (examples: #2390, #2505, #605).

Potential Capstone: MATLAB's help text has no real docs, only argument types. This ports wrap's existing Python doc-extraction into MATLAB's code generator.

How it works: One shared parser (XMLDocParser) feeds into separate code-gen per language (pybind_wrapper.py existing, matlab_wrapper.py, proposed new).

Pose3.interpolateRt(): MATLAB help today vs. with this prototype

Doxygen comment via gtsam's (gtsam/geometry/Pose3.h); MATLAB output is from this unmerged prototype.

BEFORE (verbatim) <pre>% INTERPOLATERt usage: % interpolateRt(Pose3 T, % double t) : returns Pose3 % % Doxygen can be found % at gtsam.org/doxygen/</pre>	AFTER (verbatim) <pre>% Interpolate between two poses via % individual rotation and translation % interpolation. % % The default "interpolate" method % minimizes geodesic distance on % the manifold [...] % % Args: % T: End point of interpolation. % t: A value in [0, 1].</pre>
---	--

JIAHAO: PYTHON/C++ ECOSYSTEM & NOTEBOOKS

8+ issues closed across two milestones (macOS/Linux) · investigating #1734

Fixed cross-platform bugs such as macOS RTTI mismatches, Windows import failures, stub-build breakage, a batched-triangulation API cutting Python↔C++ overhead, & more.

Capstone: GTSAM's examples are mature scripts, almost none are notebooks. Migrating them to Jupyter (runnable on Colab), allows for no local build, inline plots a script can't show.

FIRST WAVE, 7 SCRIPTS → NOTEBOOKS

GPSFactorExample · OdometryExample · SimpleRotation · VisualISAMExample + 3 more

Example (SimpleRotation.py): a trivial 1-variable, 1-factor optimization — first target since it's the smallest possible working notebook.

Next: scaling toward medium-complexity examples, each following the same loop: outline, implement, verify, PR.

OPEN SOURCE OUTCOMES

- Learned real maintainer-community conventions, not just how to write code for this project.
- Got real feedback on PR descriptions, including writing for reviewers who weren't in the room, and including exact recreation steps (OS, versions, exact repro steps, etc)
- Every change reviewed and approved before merging, which helped teach importance of guardrails, for the CI benchmark jobs, and more.
- This project introduced updating multiple connected repositories, for example merging changes not into borglab/wrap itself, wrap as GTSAM uses it as a git subtree, so the fixes don't reach it automatically
- On a project this size, a fix isn't done until it's documented well enough for the next contributor to reuse, with thorough testing.